DermAI

Skin Disease Detection System

High-Level Design Document

Course	DA5402 — MLOps | IIT Madras
Author	Harshit (da25s003)
Dataset	HAM10000 — 10,015 dermoscopy images, 7 classes
Model	EfficientNetB3 | Macro F1: 0.7423
Date	April 2026

1. Introduction

DermAI is an end-to-end MLOps application for automated skin disease classification from dermoscopy images. It implements the complete AI product lifecycle: data versioning, model training, REST API serving, user authentication, real-time monitoring, and automated alerting — all containerised and orchestrated via Docker Compose.

1.1 Business Problem

Skin cancer is one of the most common cancers globally. Early detection significantly improves treatment outcomes. DermAI provides an AI-assisted triage tool that classifies dermoscopy images into 7 diagnostic categories, enabling faster initial screening.

1.2 Success Metrics

Metric	Target	Achieved
Model Macro F1	≥ 0.70	0.7423 ✓
API Inference p50	< 4s (CPU)	~3.2s ✓

Metric	Target	Achieved
Auth Flow	OTP email verified	Implemented ✓
System Uptime	Container auto-restart	unless-stopped ✓
Monitoring	25+ live metrics	Prometheus+Grafana ✓

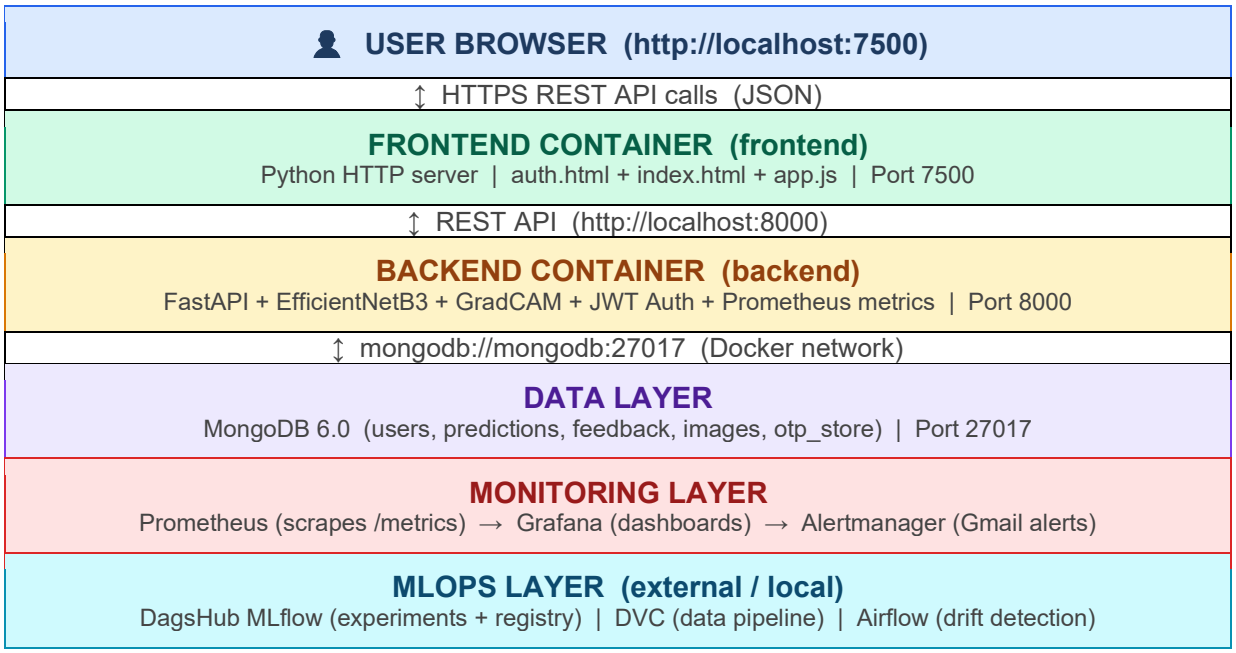
2. System Architecture

2.1 Architecture Overview

DermAI follows a loosely-coupled microservice architecture. The browser communicates exclusively with the backend via REST API calls. No frontend code has direct database access. All services run as Docker containers on a shared bridge network (dermai_net).

2.2 Architecture Diagram

The following diagram shows data flow across all system layers:



3. Component Descriptions

3.1 Frontend (dermai-frontend)

Technology: Python stdlib HTTP server (serve_frontend.py) | Static HTML/JS/CSS

Responsibility: Serves the browser-facing application. At startup, writes a runtime config.js with the API_URL pointing to localhost:8000 so the browser JavaScript knows where to send requests.

Key pages:

- auth.html — Registration, OTP verification, login
- index.html — Image upload, prediction display, GradCAM viewer, feedback

3.2 Backend (dermai-backend)

Technology: FastAPI 0.115 + Uvicorn + PyTorch (CPU) + EfficientNetB3

Responsibility: All business logic. Handles authentication, image inference, GradCAM generation, MongoDB persistence, and Prometheus metric export. Loads EfficientNetB3 weights at startup from outputs/models/best_model_cpu.pth.

Key modules:

- src/api/main.py — App wiring, background metrics thread, lifespan
- src/api/auth.py — Signup, OTP, login, user profile endpoints
- src/api/deps.py — Authorization middlewares token verification and generation
- src/api/predict.py — /predict, /explain, /feedback endpoints
- src/api/metrics.py — 25+ Prometheus Counter/Gauge/Histogram definitions
- src/db/mongodb.py — Auto-reconnect MongoDB client with lazy connection
- src/models/model.py — EfficientNetB3 builder

3.3 MongoDB (dermai-mongodb)

Image: mongo:6.0.14 | Port: 27017 | Storage: Named Docker volume (mongodb_data)

Collections:

Collection	Key Fields	Purpose
users	uid, email, username, verified…	User accounts + OTP verification
predictions	prediction_id, uid, predicted_class, confidences, image_id...	Per-user prediction history
feedback	prediction_id, comment, uid, vote…	Thumbs up/down and comment per prediction
images	image_id, prediction_id, uid, filename, image_b64, image_size(kb)…	Raw images for future retraining
otp_store	email, otp, expires_at…	TTL-indexed OTP (10 min expiry)
request_log	uid, timestamp…	Rate limiting (TTL 1 hour)

3.4 Monitoring Stack

Service	Port	Role
Prometheus v2.51.2	9090	Scrapes /metrics every 15s, evaluates alert rules

Service	Port	Role
Grafana 10.4.3	3005	Real-time dashboards from Prometheus data
Alertmanager 0.27.0	9093	Routes alerts to Gmail via SMTP
Node Exporter 1.8.2	9182	Host OS metrics (CPU, memory, disk)
MongoDB Exporter 0.40	9216	MongoDB internal metrics
Apache airflow 2.9.2-python-3.10	8080	Airflow drift detection auto triggered

3.5 MLOps Layer

- DVC: 3-stage pipeline (prepare → train → evaluate). Data and models versioned on DagsHub remote with all experiment logged onto dagshub mlflow.
- MLflow: Experiment tracking on DagsHub. Logs params, metrics, confusion matrix, GradCAM samples per run. Model registered as skin-disease-classifier@production.
- MLproject + python_env.yaml: Reproducible training environment. Any machine can run mlflow run to reproduce training, version 9 for cpu registory and version 7 model registory for gpu oriented machines.
- Airflow: Nightly DAG validates data, computes class distribution drift (PSI), alerts if retraining is needed.
- serve.sh: Launches mlflow models serve for the registered production model.

4. Key Design Decisions

Decision	Rationale
State-dict weights, not MLflow pickle	Avoids Python version incompatibility between Linux CUDA training machine and Windows CPU serving machine
API URL = localhost in frontend	Browser JS runs on host machine, not inside Docker. 'backend' hostname only resolves inside Docker network
MongoDB lazy reconnect	Backend starts before MongoDB is fully ready. _ensure_connected() retries every 5s, eliminating race conditions
Background metrics thread	Prometheus scrapes every 15s. Middleware-only metrics go stale between requests. Background thread updates CPU/memory every 5s
Prometheus v2.51 not v3.x	v3.x changed inhibit_rules schema, breaking existing rules.yml
Grafana 10.4 not 12.x	v12.x has WebSocket token rotation bug causing 401 errors
MongoDB 6.0 not 7.x	6.0 includes mongosh for healthcheck; 7.x had verbose WiredTiger logs difficult to suppress

5. Security Considerations

- Passwords hashed with SHA-256 before storage — plaintext never persisted
- JWT tokens with configurable expiry (default 60 min). Secret key stored in .env (gitignored)

- Email OTP verification prevents fake account creation. OTT expires in 10 minutes (MongoDB TTL index)
- .env is gitignored. .env.example committed with placeholder values only
- alertmanager.yml is gitignored alertmanager.yml.template is committed with placeholder values.
- Rate limiting: requests logged per user; breach triggers Prometheus alert
- CORS configured for all origins (development). Should be restricted to known domains in production

6. Deployment Architecture

All services are orchestrated via a single docker-compose.yml. The startup sequence is enforced via healthcheck conditions:

Order	Service	Waits For
1	mongodb	(starts first)
2	backend	mongodb: service_healthy
3	frontend	backend: service_healthy
3	mongodb-exporter	mongodb: service_healthy
4	alertmanager, node-exporter	(no dependency)
5	prometheus	backend, alertmanager, node-exporter, mongodb-exporter
6	grafana	prometheus: service_healthy